

P0 — rozproszone wykonanie w Ballistcie na klastrze z osobnych procesów

Sprawozdanie z weryfikacji: metoda, wyniki, znaleziska i znaczenie dla pracy

Miłosz Kowalewski

30 września 2026

1. Cel i miejsce w pracy

Metodyka pomiarów (docs/superpowers/specs/2026-09-29-metodyka-benchmarkow-design.md) stawia przed pomiarami wydajności pytanie wstępne **P0: czy wykonanie w Ballistcie jest rzeczywiście rozproszone**. Dotychczasowe wyniki pochodziły z trybu standalone, w którym scheduler i executor działają w jednym procesie: plan był serializowany i przechodził przez shuffle, ale nie przekraczał granicy procesu. Poprawność wyników dowodziła, że operacje liczą się dobrze — nie dowodziła, że liczą się w sposób rozproszony.

Zadanie P0 miało to rozstrzygnąć bez pomiarów czasu, czterema niezależnymi dowodami, na klastrze złożonym z osobnych procesów systemu operacyjnego (natywnie, bez kontenerów).

Wynik: pytanie P0 jest dla Ballisty zamknięte twierdząco.

2. Co zbudowano

Element	Rola
src/bin/ballista_node.rs	węzeł klastra jako osobny proces: rola scheduler albo executor; ścisłe parsowanie argumentów (błąd → kod 2); flaga --no-codecs dla kontroli negatywnej
src/cluster.rs	jedno źródło konfiguracji dla klienta i węzłów: kodery planu logicznego i fizycznego, konfiguracja sesji z koderami, bio-owy stan sesji
src/runner.rs	tryb zdalny klienta (BALLISTA_SCHEDULER_URL; pusta wartość = standalone) oraz katalog wyników (DIST_OUTPUT_DIR, domyślnie output)
tests/test_ballista_multi_process.py	testy uruchamiające klaster z procesów oraz 14 testów: cykl życia klastra, cztery dowody, przypadki brzegowe
tests/conftest.py,	izolacja zmiennych klienta od testów trybu standalone
tests/test_env_isolation.py	

Binarka ballista_node składa scheduler i executor z oficjalnych bibliotek Ballisty 53 (ballista-scheduler, ballista-executor) przez ich publiczne API konfiguracyjne (override_logical_codec, override_physical_codec, override_config_producer, override_session_builder). **Kod Ballisty pozostaje niezmodyfikowany** — założenie pracy „integracja jako rozszerzenie w czasie działania, bez forkowania silnika” jest zachowane.

3. Co musiało się zmienić względem trybu standalone

1. **Bio-owy stan sesji po stronie schedulera.** W trybie standalone scheduler dostaje gotowy stan sesji klienta (`new_standalone_scheduler_from_state`) — z bio-owym planistą, regułami optymalizatora i koderami. Osobny proces nie ma dostępu do pamięci klienta, więc scheduler buduje ten stan sam (`cluster::bio_session_state`); bez tego plan fizyczny powstałby bez reguł polars-bio.
2. **Kodery rejestrowane w każdym procesie osobno** — klient, scheduler, executor.
3. **Przestrzeń nazw ballista w konfiguracji schedulera** (`upgrade_for_ballista()`). Bez niej scheduler cicho odrzuca wszystkie klucze ballista.* wysłane przez klienta (log na poziomie debug: „Could not find config namespace “ballista””), więc strojenie opcji Ballisty nie miałyby żadnego skutku. Wykryte w przeglądzie kodu, potwierdzone testem.
4. **Szeregowanie push z rozdziałem round-robin.** W trybie pull executory same pobierają zadania; przy milisekundowych zadaniach jeden executor potrafi zgarnąć całą pracę, a dowód „praca na obu executorach” stałby się loterią.
5. **Wyłączone sprzątanie danych zakończonych zadań** (`finished_job_data_clean_up_interval_seconds = 0`) — pliki etapów w katalogach roboczych executorów są dowodem.

4. Metoda dowodu

#	Dowód	Jak sprawdzany
1	osobne procesy	różne PID-y schedulera, dwóch executorów i klienta; scheduler widzi dwa executory o różnych identyfikatorach i portach (REST: <code>/api/executors</code>)
2	poprawność	wynik każdej z pięciu operacji porównany z wyrocznią polars-bio (te same wyrocznie co testy standalone)
3	praca na obu executorach	każdy executor ma własny katalog roboczy; Ballista zapisuje tam dane każdego policzonego etapu (<code>work_dir/job_id/stage_id/...</code>), więc katalogi pokazują wprost, kto co liczył; czytnik shuffle odczytuje lokalnie tylko pliki z własnego katalogu, więc dane etapu z drugiego executora muszą przejść przez sieć (Arrow Flight)
4	kontrola negatywna	executor uruchomiony z <code>--no-codecs</code> ; oczekiwane odrzucenie planu przy dekodowaniu

Dowody 1-3 sprawdza jeden test sparametryzowany pięcioma operacjami; dowód 4 — osobny test, zweryfikowany mutacyjnie (ten sam test z executorem mającym kodery kończy się porażką).

5. Wyniki

Każda operacja to dwa zapytania do klastra (wynik i EXPLAIN ANALYZE). W komórkach — numery etapów, dla których dany executor zapisał dane:

Operacja	executor_1	executor_2
overlap	zapytanie 1: 1; zapytanie 2: 1	—
merge	zapytanie 1: 1, 2, 3; zapytanie 2: 1, 2, 3	zapytanie 1: 1, 2; zapytanie 2: 1, 2
subtract	zapytanie 1: 1, 2, 3, 4; zapytanie 2: 1, 2, 3, 4	zapytanie 1: 1, 2, 3; zapytanie 2: 1, 2, 3

Operacja	executor_1	executor_2
nearest	zapytanie 1: 1, 2; zapytanie 2: 1, 2	zapytanie 1: 1; zapytanie 2: 1
coverage	zapytanie 1: 1, 2; zapytanie 2: 1, 2	zapytanie 1: 1; zapytanie 2: 1

- **merge, subtract:** oba executory liczyły etap 1 (zapis shuffle po chromosomie) i etapy kolejne — dane musiały przejść między procesami.
- **nearest, coverage:** praca rozłożona na oba executory (wzorzec broadcast).
- **overlap:** jedno zadanie na etap (pojedyncze pliki wejściowe, brak równoległości hash — ograniczenie znane z wcześniejszego etapu); etap wykonuje się w procesie executora, ale na jednym.
- **Kontrola negatywna:** executor bez koderów liczy etap 1 (zwykle węzły DataFusion), ale etapu z węzłem MergeExec nie potrafi zdekodować i zwraca schedulerowi `InvalidArgument: Could not deserialize BallistaPhysicalPlanNode: failed to decode Protobuf message`. Plan jest więc dekodowany w executorze.

Pakiet testów: **57/57** (42 wcześniejsze + 15 nowych), także z celowo wyeksportowanymi zmiennymi `BALLISTA_SCHEDULER_URL` i `DIST_OUTPUT_DIR` (izolacja).

6. Znaleziska

1. **Odporność Ballisty na błąd dekodowania planu.** W trybie push błąd dekodowania przy uruchamianiu zadania jest traktowany jak utrata executora, a nie błąd zadania: scheduler wyrejestrowuje executor, a po jego ponownej rejestracji (co ok. 60 s) ponawia — zapytanie **wisi zamiast zakończyć się błędem**. To istotna obserwacja dla kryterium jakościowego „odporność i diagnostyka” w porównaniu silników.
2. **Executor łączy się ze schedulerem tylko raz przy starcie** (`scheduler_connect_timeout_seconds = 0` oznacza jedną próbę; wartość dodatnia uruchamia pętlę, której limit w Ballistcie 53 nigdy nie wygasa). Kolejność startu ma znaczenie — scheduler musi nasłuchiwać pierwszy.
3. **Tryb zdalny planuje końcowe sortowanie overlap w jednym etapie**, standalone w dwóch (osobny etap `SortPreservingMergeExec`). Złączenie `IntervalJoinExec` w obu trybach wykonuje się na executorze; pozostałe operacje mają w obu trybach tę samą liczbę etapów (merge 3, subtract 4, nearest 2, coverage 2).
4. **Cicho odrzucane opcje ballista.*** bez `upgrade_for_ballista()` (sekcja 3, pkt 3) — pułapka, która w pomiarach unieważniałaby strojenie Ballisty bez żadnego sygnału.
5. **Wspólne pliki wyników jako źródło zależności między testami** — uruchomienia zdalne nadpisywały plan, na którym opierał się test trybu standalone; rozwiązane katalogiem wyników (`DIST_OUTPUT_DIR`) i izolacją zmiennych klienta.

7. Znaczenie dla pracy magisterskiej

- **Potwierdzenie założenia o braku forka w realnym środowisku rozproszonym.** Kodery i funkcje tabelowe działają, gdy klient, scheduler i executory nie współdzielą pamięci.
- **Warunek wstępny pomiarów spełniony.** Pomiary P1-P3 dla Ballisty zostaną wykonane na klastrze z procesów z emulacją węzłów — bez P0 teza o rozproszeniu byłaby niepoparta.
- **Wkład metodyczny.** Czteroczęściowy sposób dowodzenia rozproszenia (osobne procesy, poprawność, ślady pracy w katalogach executorów, kontrola negatywna) można opisać jako element rozdziału o weryfikacji; typowo pokazuje się jedynie poprawność wyniku.
- **Materiał do porównania silników.** Wymagania operacyjne (samodzielna konfiguracja schedulera, kolejność startu, tryb push dla deterministycznego rozkładu) i odporność

(zawieszenie przy błędzie dekodowania) zasilają jakościowe kryteria porównania Ballisty i Saila.

- **Kandydat do dalszej pracy:** overlap bez równoległości — łączy się z pytaniem P3 (algorytmy) i z trybem Partitioned złączenia interwałowego.

8. Ograniczenia

- Jedna maszyna: brak fizycznej sieci (ruch przez interfejs pętli zwrotnej).
- Wspólny system plików: ładunek planu przenosi ścieżki do danych; w kontenerach i w chmurze potrzebny wspólny magazyn, a ballista_node wiąże na sztywno 127.0.0.1/localhost.
- Dane testowe to pięć przedziałów na tabelę — brak jakichkolwiek wniosków o wydajności.
- Sail nadal działa lokalnie w jednym procesie (asymetria do rozstrzygnięcia próbą K8s).

9. Uwagi odłożone z przeglądu kodu

Przegląd całej gałęzi (jeden niezależny recenzent) nie wykazał błędów krytycznych; trzy uwagi istotne zostały poprawione (wyścig przy starcie klastra, wyciek zmiennych do testów standalone, niespójność specyfikacji dla overlap), a uwaga o odrzucanych opcjach ballista.* — dodatkowo po przeglądzie. Odłożone, drobne: dowód 3 liczony łącznie dla obu zapytań; zbyt ogólny wzorzec komunikatu w kontroli negatywnej; nieaktualny docstring modułu testów; niespójne podanie czasu w komentarzu testu; zbyt optymistyczny komentarz w docker-compose; test martwego schedulera sprawdza tylko kod wyjścia; komunikat „Klaster Ballista gotowy.” przed faktycznym połączeniem; drobne kwestie odporności pomocników testowych; zależność cykliczna modułów cluster i runner; zależność testu dowodów standalone od kolejności uruchomień (sprzed P0).

10. Kolejne kroki

1. Dane: pobranie zestawu databio-8p i przejście z CSV na Parquet w obu ścieżkach.
2. Narzędzie pomiarowe (metryki, suma kontrolna, runnery, orkiestrator, smoke), w tym podniesienie limitów gRPC w ballista_node.
3. Próba Saila na Kubernetesie.